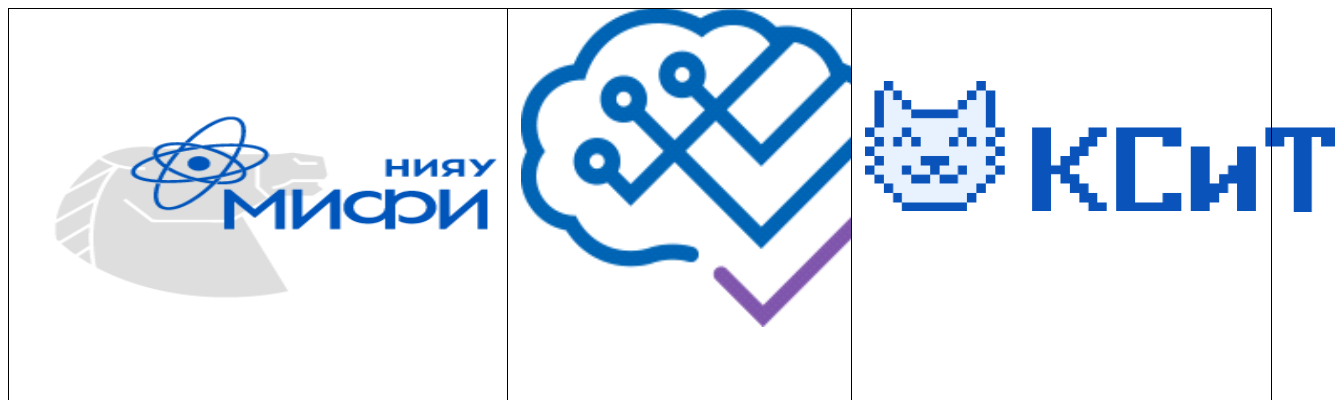


Национальный исследовательский ядерный университет «МИФИ»

Институт интеллектуальных кибернетических систем

Кафедра №12 «Компьютерные системы и технологии»



ОТЧЕТ

О выполнении лабораторной работы №6 «Работа со структурами данных на основе списков»

Студент:	Сейтназаров Д.Н
Группа:	Б24-703
Преподаватель:	Храпов А.С

Москва – 2024

- **Формулировка индивидуального задания**

- Вариант №2: Удалить все символы в строке, стоящие до первой точки и после последней точки с запятой

- **Описание использованных типов данных**

При выполнении данной лабораторной работы использовался встроенный тип данных `int`, предназначенный для работы с целыми числами.

- **Описание использованного алгоритма**

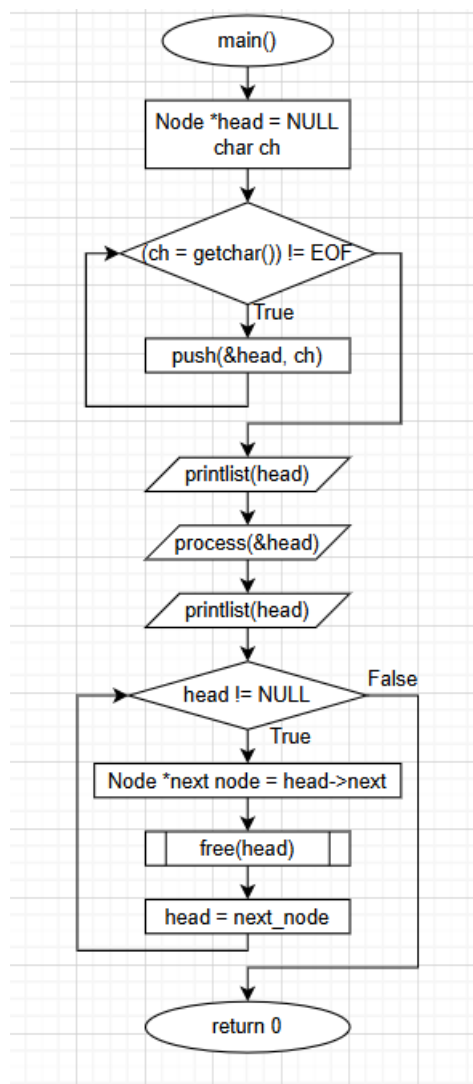


Рис. 1: Блок-схема алгоритма работы функции `main()`

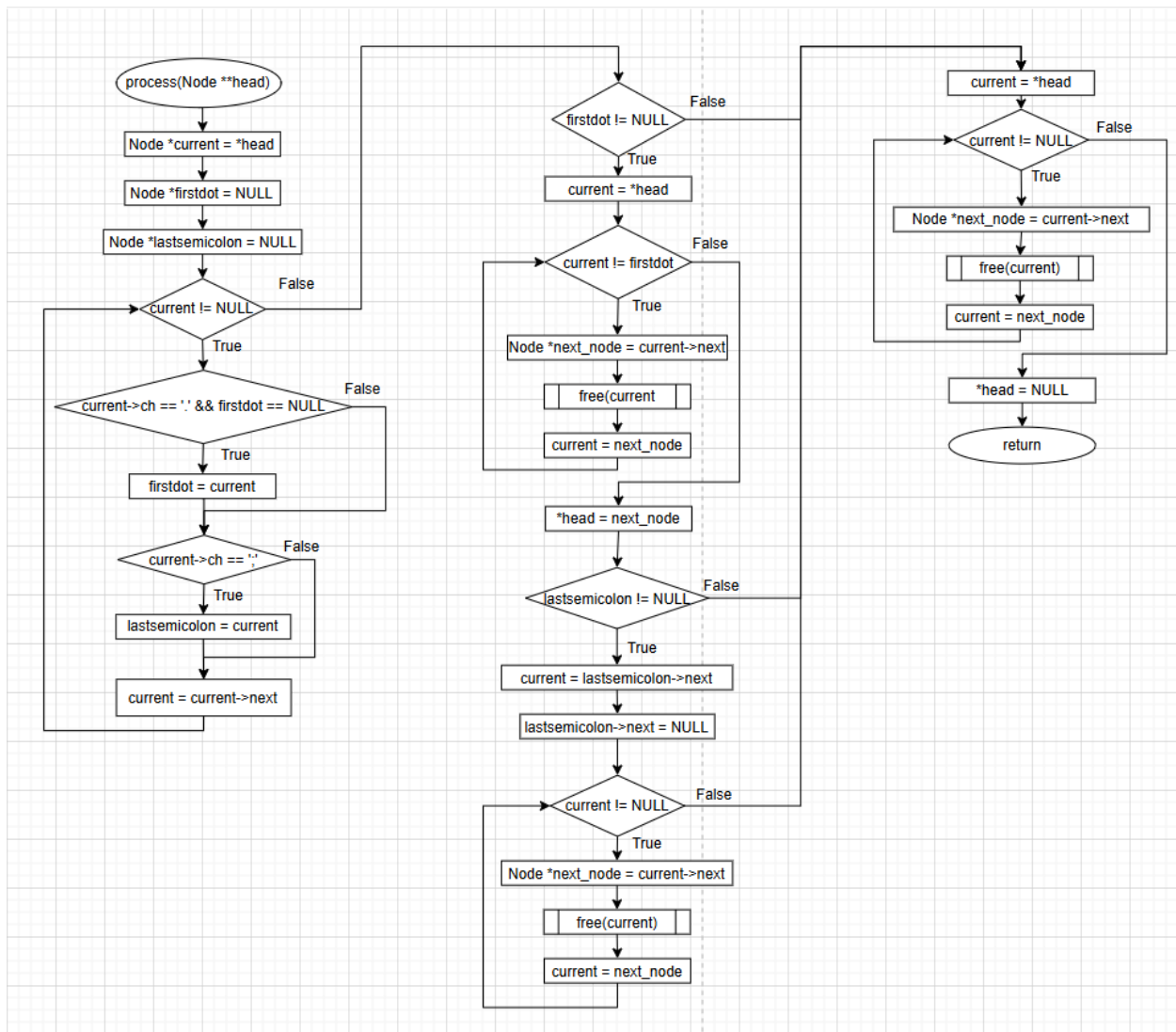


Рис. 2: Блок-схема алгоритма работы функции *process (Node **head)*

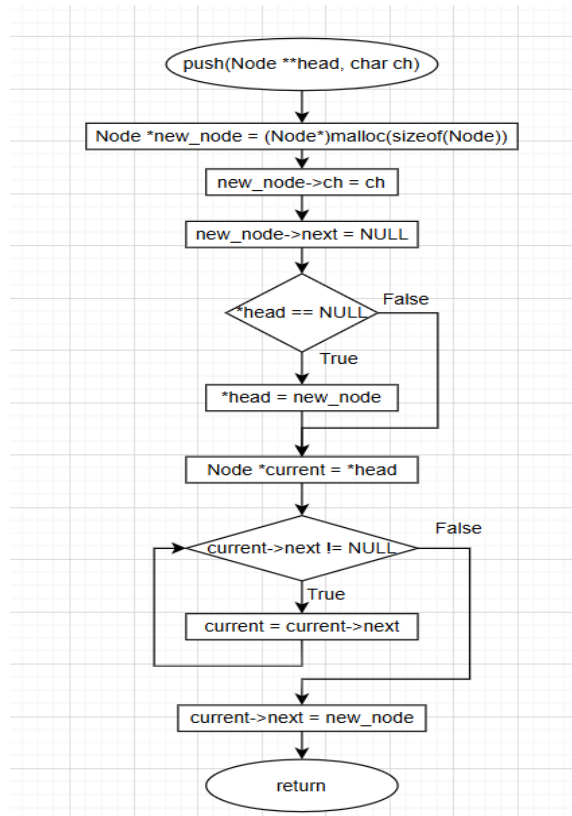


Рис. 3: Блок-схема алгоритма работы функции `push(Node **head, char ch)`

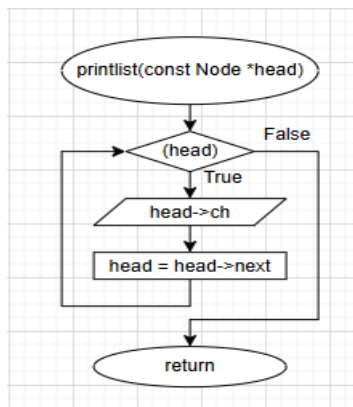


Рис. 4: Блок-схема алгоритма работы функции `printlist(const Node *head)`

• Исходные коды разработанных программ

Листинг 1: Исходные коды программы lab6 (файл: lab6.c)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct Node {
5     char ch;
6     struct Node *next;
7 } Node;
8
9 void push(Node **head, char ch) {
10     Node *new_node = (Node*)malloc(sizeof(Node));
11     new_node->ch = ch; // значение ноды
12     new_node->next = NULL;
13     if (*head == NULL) {
14         *head = new_node;
15     }
16     else {
17         Node *current = *head;
18         while (current->next != NULL) {
19             current = current->next;
20         }
21         current->next = new_node;
22     }
23 }
24
25 void printlist(const Node *head) {
26     while (head) {
27         printf("%c", head->ch);
28         head = head->next;
29     }
30     printf("\n");
31 }
32
33 void process(Node **head) {
34     Node *current = *head;
35     Node *firstdot = NULL;
36     Node *lastsemicolon = NULL;
37     while (current != NULL) {
38         if (current->ch == '.' && firstdot == NULL) {
39             firstdot = current;
40         }
41         if (current->ch == ';') {
42             lastsemicolon = current;
43         }
44         current = current->next;
45     }
46     if (firstdot != NULL) {
47         current = *head;
48         while (current != firstdot) {
```

```

49         Node *next_node = current->next;
50         free(current);
51         current = next_node;
52     }
53     *head = firstdot;
54     if (lastsemicolon != NULL) {
55         current = lastsemicolon->next;
56         lastsemicolon->next = NULL;
57         while (current != NULL) {
58             Node *next_node = current->next;
59             free(current);
60             current = next_node;
61         }
62     }
63 }
64 else {
65     current = *head;
66     while (current != NULL) {
67         Node *next_node = current->next;
68         free(current);
69         current = next_node;
70     }
71     *head = NULL;
72 }
73 }
74
75 int main() {
76     Node *head = NULL;
77     char ch;
78
79     while ((ch = getchar()) != EOF) {
80         push(&head, ch);
81     }
82
83     printf("\n");
84     printf("Исходная строка: \n");
85     printlist(head);
86
87     process(&head);
88
89     printf("Модифицированная строка: \n");
90     printlist(head);
91     printf("\n");
92
93     while (head != NULL) {
94         Node *next_node = head->next;
95         free(head);
96         head = next_node;
97     }
98     return 0;
99 }

```

- **Описание тестовых примеров**

Таблица 1: Тестовые примеры

	input	output
Пример 1	123qwe..asdzxc;zxczxc	..asdzxc;
Пример 2	asdasdasdqwe...qwe...qwe;asdzxc;qwerty	...qwe...qwe;asdzxc;

- **Скриншоты**

```
[seitnazarov.dn@unix lab6]$ ls
file lab6.c
[seitnazarov.dn@unix lab6]$ gcc lab6.c -o file
[seitnazarov.dn@unix lab6]$ valgrind ./file
==19305== Memcheck, a memory error detector
==19305== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==19305== Using Valgrind-3.24.0 and LibVEX; rerun with -h for copyright info
==19305== Command: ./file
==19305==
123qwe..asdzxc;zxczxc

Исходная строка:
123qwe..asdzxc;zxczxc

Модифицированная строка:
..asdzxc;

==19305==
==19305== HEAP SUMMARY:
==19305==    in use at exit: 0 bytes in 0 blocks
==19305==   total heap usage: 24 allocs, 24 frees, 2,400 bytes allocated
==19305==
==19305== All heap blocks were freed -- no leaks are possible
==19305==
==19305== For lists of detected and suppressed errors, rerun with: -s
==19305== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Рис. 5: Сборка и запуск программы lab6.c

```

[seitnazarov.dn@unix lab6]$ valgrind ./file
==19418== Memcheck, a memory error detector
==19418== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==19418== Using Valgrind-3.24.0 and LibVEX; rerun with -h for copyright info
==19418== Command: ./file
==19418==
asdasdasdqwe...qwe...qwe;asdzxc;qwerty

Исходная строка:
asdasdasdqwe...qwe...qwe;asdzxc;qwerty

Модифицированная строка:
...qwe...qwe;asdzxc;

==19418==
==19418== HEAP SUMMARY:
==19418==      in use at exit: 0 bytes in 0 blocks
==19418==    total heap usage: 41 allocs, 41 frees, 2,672 bytes allocated
==19418==
==19418== All heap blocks were freed -- no leaks are possible
==19418==
==19418== For lists of detected and suppressed errors, rerun with: -s
==19418== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
[seitnazarov.dn@unix lab6]$ |

```

Рис. 6: Запуск программы для тестового примера

• Выводы

В ходе выполнения лабораторной работы № 6 «Работа со структурами данных на основе списков» была разработана программа, которая осуществляет обработку строковых данных, представленных в виде односвязного списка. Основные цели:

- Работа со структурами данных на основе списков
- Чтение данных с помощью `getchar()`
- Создание новых узлов в списке / Удаление узлов из списка
- Очистка списка в конце программы